NYC Taxi Data Analytics Platform

Big Data Storage and Processing

Dao Vu Tien Dat Tran Huu Dao Ngo Quang Duc Vu Nhu Duc Ta Tuan Hai

School of Information and Communication Technology
Course: IT4043E

December 2025

Mentor: PhD. Tran Viet Trung

Outline

1. Introduction
2. Data Model & Architecture
3. Data Ingestion Layer
4. Data Storage Layer
5. Analytics & Serving Layer
6. Visualization Dashboard

Problem Statement

Challenge: Bridge the gap between raw taxi trip data and actionable analytics

Key Problems:

- Massive volumes of heterogeneous trip data
- Need for real-time and batch processing
- Complex spatio-temporal analysis requirements
- Integration of multiple data systems

Goal: Build an end-to-end big data pipeline for NYC taxi analytics with:

- Scalable data ingestion
- Medallion architecture (Bronze/Silver/Gold)
- Interactive visualizations
- Real-time monitoring

Motivation

Why NYC Taxi Data?

- Rich, publicly available dataset from NYC TLC
- Natural embodiment of Big Data characteristics (Volume, Velocity, Variety)
- Real-world relevance: transportation planning, pricing, demand forecasting

Platform Objectives:

1. Design event-driven data architecture with CDC and streaming
2. Implement Medallion pattern for progressive data refinement
3. Deliver interactive business intelligence dashboards
4. Enable data-driven decision making for fleet management

NYC TLC Trip Record Schema

Temporal Fields

- tpep_pickup_datetime
- tpep_dropoff_datetime

Spatial Fields

- PULocationID (Pickup Zone)
- DOLocationID (Dropoff Zone)

Trip Metrics

- passenger_count
- trip_distance

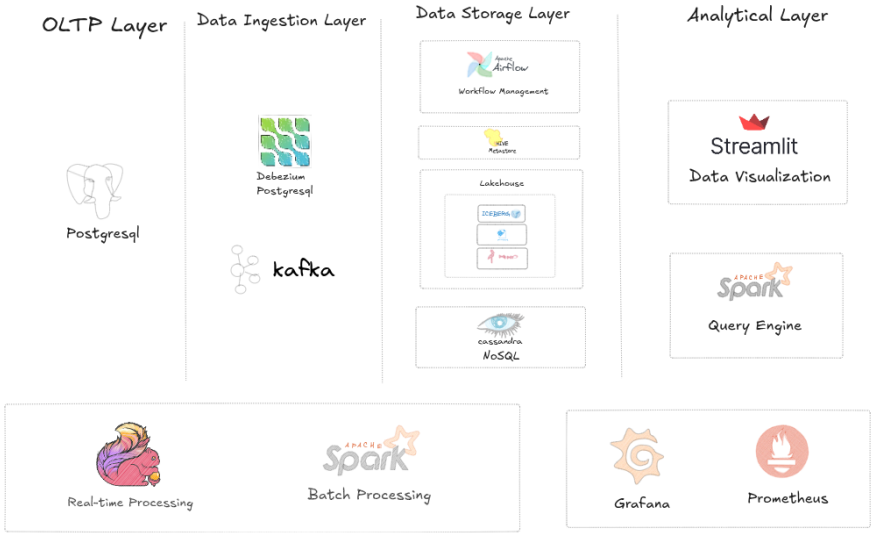
Fare Components

- fare_amount
- tip_amount
- tolls_amount
- total_amount

Payment & Other

- payment_type
- RatecodeID
- VendorID
- congestion_surcharge

System Architecture



Key Technologies

Ingestion Layer

- PostgreSQL (OLTP)
- Debezium (CDC)
- Apache Kafka

Storage Layer

- MinIO (S3)
- Apache Iceberg
- Apache Cassandra

Processing

- Apache Spark
- Spark Thrift Server
- Apache Flink

Orchestration

- Apache Airflow
- Docker Compose

Analytics

- Hive Metastore
- Streamlit + PyHive
- Plotly Charts

Monitoring

- Prometheus
- Grafana

Medallion Architecture

Bronze Layer

- Raw CDC events
- Kafka topic data
- No transformations

Silver Layer

- Deduplicated records
- Schema validation
- Type conversions

Gold Layer

- Fact & Dimension tables
- Business aggregates
- Dashboard-ready

PostgreSQL (OLTP Layer)

Configuration:

- PostgreSQL 16-alpine
- WAL level: `logical` (for CDC)
- Database: `nyc_taxi_db`

Data Simulator:

- Python-based generator
- Realistic NYC coordinates
- 10 records every 5 seconds
- Historical + real-time data

Schema: taxi_trips

```
CREATE TABLE taxi_trips (
  id SERIAL PRIMARY KEY,
  tpep_pickup_datetime TIMESTAMP,
  tpep_dropoff_datetime TIMESTAMP,
  passenger_count INTEGER,
  trip_distance FLOAT,
  pickup_longitude FLOAT,
  pickup_latitude FLOAT,
  fare_amount FLOAT,
  tip_amount FLOAT,
  total_amount FLOAT,
  payment_type INTEGER
);
```

Debezium - Change Data Capture

CDC Pipeline:

1. PostgreSQL writes to WAL (`wal_level=logical`)
2. Debezium reads from replication slot
3. Events transformed to JSON
4. Published to Kafka topic: `taxi.public.taxi_trips`

Key Configurations:

- Plugin: `pgoutput` (native replication)
- Snapshot mode: `initial`
- Transform: `ExtractNewRecordState`

Benefits:

- Zero impact on OLTP performance
- Exactly-once delivery semantics
- Real-time sync with analytics
- Automatic schema evolution

Connector:

- `debezium/connect:2.5`
- Auto-registered via init container
- JSON converters (no schema)

Apache Kafka - Event Streaming

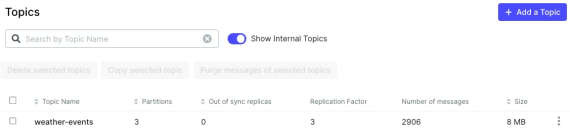
Configuration:

- KRaft mode (no Zookeeper)
- Single broker for development
- Auto topic creation enabled

Topics:

- `taxi.public.taxi_trips`
- `debezium_configs`
- `debezium_offsets`
- `debezium_statuses`

Kafka UI: Port 8080 for monitoring



Kafka UI: Topic Details

MinIO - Data Lakehouse Storage

Configuration:

- S3-compatible object storage
- API Port: 9000, Console: 9001
- Bucket: `lakehouse`

Integration:

- Spark via S3A filesystem
- Path-style access enabled
- Iceberg table format

Spark S3A Config:

```
spark.hadoop.fs.s3a.endpoint=
    http://minio:9000
spark.hadoop.fs.s3a.access.key=admin
spark.hadoop.fs.s3a.secret.key=12345678
spark.hadoop.fs.s3a.path.style.access=true
```

Bucket Initialization:

- Auto-created via `mc` container
- Public read policy applied

Spark Medallion Processing - Bronze Layer

Bronze — Ingest and Preserve

- Ingests raw Kafka/Debezium CDC events with minimal transformation
- Preserves original payload fields (typically as strings or raw types)
- Includes ingestion metadata (timestamps, Kafka offsets) for latency tracing
- Runs as streaming job with checkpointed `foreachBatch` writes
- Append-only design maintains a faithful event trail

```
-- Example Bronze table (Iceberg)
CREATE TABLE IF NOT EXISTS lakehouse.bronze.taxi_trips (
    id INTEGER,
    tpep_pickup_datetime STRING,
    tpep_dropoff_datetime STRING,
    passenger_count STRING,
    trip_distance STRING,
    ...
) USING iceberg;
```

Spark Medallion Processing - Silver Layer

Silver — Clean, Normalize, and Enrich

- Converts timestamp strings to `TIMESTAMP`, casts numerics
- Applies UDF mappings (e.g., payment code → label)
- Computes derived metrics (trip duration)
- Filters invalid rows, fills/defaults missing values, deduplicates
- Runs incrementally using `watermarks` table (`last_watermark_ts`)

```
-- Watermark table for incremental processing
CREATE TABLE IF NOT EXISTS lakehouse.bronze.watermarks (
    table_name STRING, last_watermark_ts LONG, updated_at TIMESTAMP
) USING iceberg;

-- Incremental filter used by Silver/Gold
SELECT * FROM lakehouse.bronze.taxi_trips
WHERE CAST(tpep_pickup_datetime AS LONG) > '<last_watermark_ts>';
```

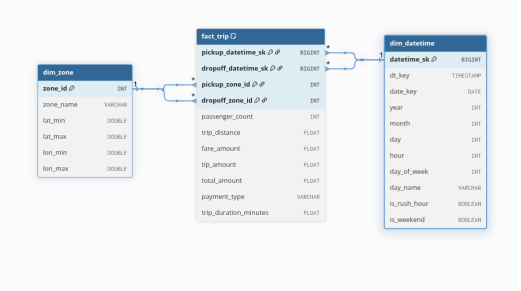
Spark Medallion Processing - Gold Layer

Gold — Dimensional Modeling and Aggregates Curated Analytics Assets:

- **Dimensions:** `dim_datetime`, `dim_zone` with surrogate keys via MERGE/upsert
- **Fact table:** `fact_trip` with FKs to dimensions, typed numerics, business fields

Iceberg MERGE for Idempotent Upserts:

```
MERGE INTO lakehouse.gold.fact_trip AS target
USING fact_trip_temp AS source
ON target.pickup_datetime_sk = source.pickup_datetime_sk
   AND target.dropoff_datetime_sk = source.dropoff_datetime_sk
   AND target.pickup_zone_id = source.pickup_zone_id
WHEN NOT MATCHED THEN INSERT *;
```



Gold Layer Data Model

Flink Streaming → Cassandra

Pipeline: Kafka (CDC) → Flink → Cassandra

Real-time KPIs:

- **Revenue per window:** Tumbling 10s window aggregates total revenue & trip count
- **Active trips:** Sliding window (2 min size, 30s slide) counts recent trips

Flink Configuration:

- JobManager + TaskManager (2 slots)
- Bounded out-of-orderness watermarks
- Web UI: Port 8081

Cassandra Setup:

- Cassandra 4.1, CQL Port: 9042
- Cluster: TaxiCluster
- Keyspace: `realtime`

Schema (Realtime KPIs):

```
CREATE TABLE realtime.revenue_minute (
  window_start TIMESTAMP PRIMARY KEY,
  window_end TIMESTAMP,
  total_revenue DOUBLE,
  trip_count BIGINT
);

CREATE TABLE realtime.active_trips (
  window_start TIMESTAMP PRIMARY KEY,
  window_end TIMESTAMP,
  active_trips BIGINT
);
```

Spark Thrift Server - JDBC Gateway

Architecture:

- HiveServer2 compatible (Port 10000)
- Connects to Hive Metastore
- Queries Iceberg tables on MinIO

Clients:

- Streamlit + PyHive
- Apache Superset
- Tableau, DBeaver

Dependencies (Runtime):

- `hadoop-aws-3.3.4.jar`
- `aws-java-sdk-bundle`
- `delta-spark_2.12-3.3.1`
- `delta-hive_2.12-3.3.1`

Connection:

```
conn = hive.Connection(
    host="spark-thrift",
    port=10000,
    username="hive"
)
```

Apache Airflow - Workflow Orchestration

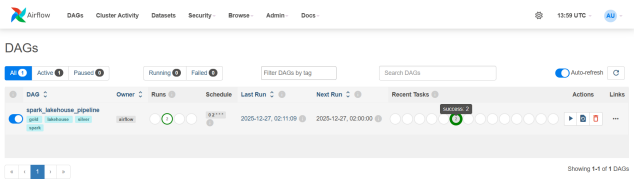
ETL Pipeline DAG:

1. `transform_silver`
2. `aggregate_gold`
3. `update_dimensions`

Configuration:

- LocalExecutor
- PostgreSQL backend
- Docker-in-Docker for Spark

Schedule: Periodic ETL runs



Airflow DAG UI

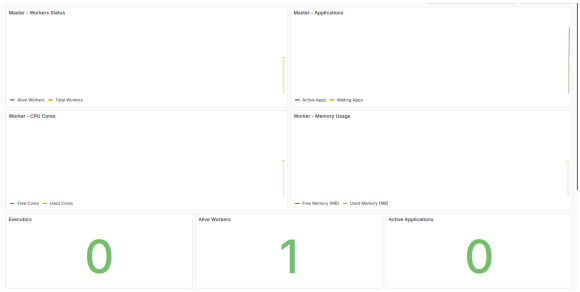
Monitoring - Prometheus & Grafana

Prometheus Metrics:

- Spark Master/Worker status
- CPU cores (free/used)
- Memory utilization
- Active applications & executors

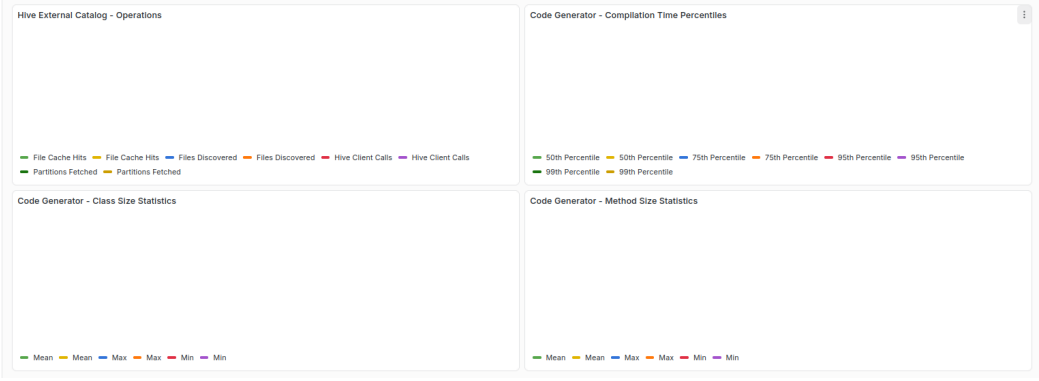
Grafana Dashboard Panels:

- Cluster health & resource utilization
- Hive catalog operations
- Code generation metrics (P50-P99)
- Generated class/method sizes



Cluster Health & Resource Panels

Grafana Dashboard - Advanced Metrics



Advanced Panels: Hive External Catalog operations, Code Generator compilation time percentiles, generated class/method size statistics

Streamlit + PyHive Architecture

Technology Stack:

- Streamlit 1.40.0
- PyHive (Hive JDBC)
- Plotly for charts
- Pandas for data handling

Connection Flow:

1. Streamlit app starts
2. PyHive connects to Spark Thrift
3. SQL queries Gold layer tables
4. Results rendered as Plotly charts

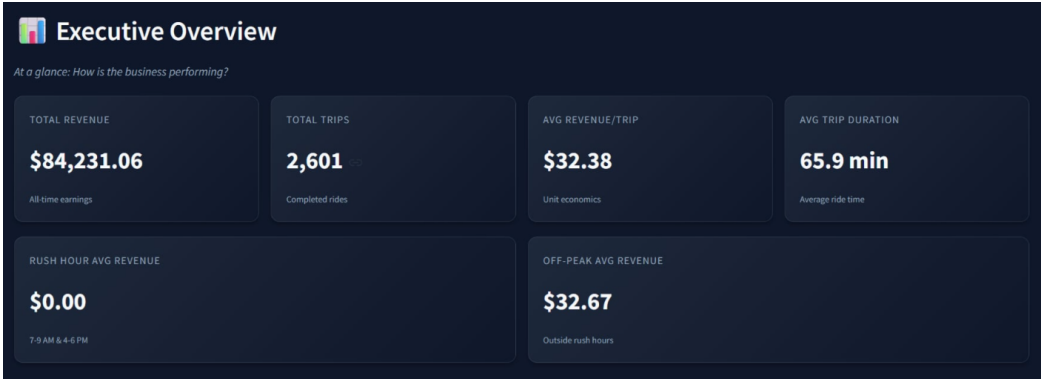
Dashboard Sections:

1. Executive Overview (KPIs)
2. Revenue Deep-Dive
3. Operations & Efficiency
4. Geographic Intelligence

Features:

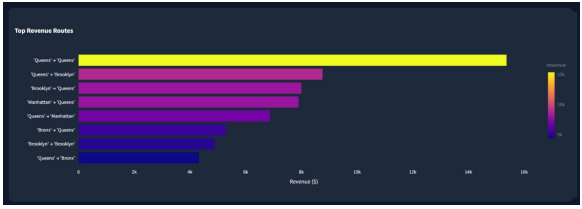
- Dark theme design
- 5-minute query caching
- Interactive filters
- Responsive layout

Executive Overview



Key Metrics: Total Revenue, Trip Count, Average Revenue/Trip, Rush Hour Comparison

Revenue Analysis & Operations



Top Revenue Routes

Operations & Efficiency

- **Revenue:** Manhattan routes dominate revenue generation
- **Tips:** Credit card payments yield 2x higher tips
- **Duration:** Most trips complete in 10-30 minutes

Geographic Intelligence

Insights: Zone-to-zone trip flow heatmap and revenue by pickup zone

Team Contributions

Member	Responsibilities	%
Tran Huu Dao	Debezium CDC (PostgreSQL → Kafka), Spark Thrift Server, Streamlit dashboards with PyHive	20%
Dao Vu Tien Dat	Batch streaming (Kafka → Iceberg), Real-time streaming in Apache Flink	20%
Ngo Quang Duc	Spark Medallion ETL scripts, Incremental processing strategy, OLAP data model design	20%
Vu Nhu Duc	Airflow DAGs for daily Lakehouse ETL, Programmatic Spark connectivity via Airflow API	20%
Ta Tuan Hai	PostgreSQL data streaming (OLTP), Grafana + Prometheus monitoring, Presentation slides	20%

Thank You!

Questions & Discussion

NYC Taxi Data Analytics Platform
Big Data Storage and Processing - IT4043E